



OPEN SOURCE ENGINEERING

Name: A SAI TEJA

ID: 2400040369

Semester: Odd

Academic Year: 2025-2026

Course Code: 24CS02EF

Under the guidance of

Dr. Arunekumar Bala

1 Understanding the Core Ubuntu Linux Distribution

1.0.1 1. Overview and Philosophy

Ubuntu is a powerful, free, and open-source operating system built upon the stable foundation of Debian Linux. It stands as the world's most popular Linux distribution for desktop use, successfully blending cutting-edge features with unparalleled user-friendliness. Developed and maintained by Canonical Ltd., Ubuntu's guiding principle is "Linux for human beings." This philosophy drives its commitment to accessibility, stability, and providing an intuitive computing experience for everyone, from novice users to seasoned developers.

1.0.2 2. The Desktop Experience (GNOME)

The standard Ubuntu desktop utilizes the **GNOME** desktop environment, which presents a modern, clean, and highly efficient graphical interface. The key design elements include a permanent dock (launcher) on the left side for quick access to essential applications, and the **Activities Overview**. This view, easily accessed by pressing the Super (Windows) key, provides a centralized hub for managing all open windows, workspaces, and system-wide searching. This streamlined workflow makes Ubuntu feel contemporary and ensures high productivity. Furthermore, Ubuntu is recognized for its strong, out-of-the-box hardware detection and compatibility, simplifying the setup process for most users.

1.0.3 3. Software Management and Packaging

Ubuntu employs a robust dual-system for software management. The traditional and reliable **Advanced Packaging Tool (APT)** manages **DEB** packages, handling core system utilities and standard applications sourced from official repositories. Complementing this is the use of **Snaps**, a modern, containerized package format pioneered by Canonical. Snaps bundle an application with all its required dependencies, guaranteeing consistent performance across different Ubuntu versions. Crucially, Snaps run in a **sandboxed** environment, isolating them from the rest of the operating system to significantly enhance overall application security. This flexibility ensures users have access to a vast, up-to-date, and secure software library.

2 Encryption and GPG

2.1 Types of Encryption in Ubuntu

Ubuntu offers two primary approaches to encryption: **Full Disk Encryption (FDE)** and **File/Directory Encryption**.

2.1.1 1. Full Disk Encryption (FDE)

- **What it is:** FDE encrypts the entire hard drive or a large partition, including the operating system files, swap space, and user directories.
- **How it works:** Ubuntu uses **LUKS** (Linux Unified Key Setup) for FDE. When the system boots, you are prompted for a **passphrase**. If correct, LUKS decrypts the entire drive, and the decryption process runs transparently in the background while the system is in use.
- **Purpose:** The primary defense against data loss due to **theft** or **physical access** to the computer when it is turned off. If someone steals the hard drive, the data is useless without the LUKS passphrase.
- **Implementation:** FDE is typically enabled during the Ubuntu installation process by selecting the "Encrypt the new Ubuntu installation" option. It's much more difficult to enable after installation.

2.1.2 2. File and Directory Encryption

- **What it is:** This method encrypts specific files, directories, or messages, offering granular control over which data is protected.
- **Tools:**
 - **GPG (GNU Privacy Guard):** The standard, used for encrypting individual files and especially for secure communication using **public-key cryptography**.
 - **eCryptfs (older):** Previously used for encrypting the user's Home directory, but has been largely phased out for FDE.

2.2 GPG (GNU Privacy Guard) Explained

GPG is the GNU implementation of the **OpenPGP** standard (originally Pretty Good Privacy - PGP). It is essential for protecting individual files and ensuring secure, authenticated communication.

2.2.1 1. Core GPG Concepts

GPG relies on **asymmetric cryptography**, which uses a pair of mathematically linked keys:

- **Public Key:** This key is shared with everyone. It can be used to **encrypt** a message that only you can read, or to **verify** a signature you created.
- **Private (Secret) Key:** This key is kept **secret** and is protected by a strong passphrase. It is used to **decrypt** messages sent to you, or to **digitally sign** files to prove they came from you.

2.2.2 2. Basic GPG Command-Line Usage

GPG is usually pre-installed on Ubuntu and is primarily used through the command line (Terminal).

A. Generating a Key Pair The first step is to create your public and private key pair:

Bash

```
gpg --full-generate-key
```

You will be prompted to select the key type (RSA and RSA is common), keysize (4096 is recommended), expiration date, and your Real Name, Email, and a strong **passphrase** to protect your private key.

B. Encrypting a File for Yourself (Symmetric Encryption) To quickly encrypt a file using a single passphrase (like a standard password), use symmetric encryption:

Bash

```
gpg -c myfile.txt
```

This command will prompt you for a passphrase and create an encrypted file named `myfile.txt.gpg`.

C. Encrypting a File for Someone Else (Asymmetric Encryption) To securely send a file, you must use the recipient's **Public Key** (which you must have previously imported into your keyring with `gpg --import`):

Bash

```
gpg --encrypt --recipient "recipient@example.com" mysecretfile.doc
```

This creates `mysecretfile.doc.gpg`. Only the recipient, who holds the corresponding Private Key, can decrypt it.

D. Decrypting a File To decrypt a file that was encrypted for you:

Bash

```
gpg --decrypt mysecretfile.doc.gpg
```

You will be prompted for the passphrase that protects your Private Key. You can use the `--output` option to specify the decrypted

3 Sending Encrypted Email

3.1 Prerequisite: Setting Up GPG

Before you can send or receive encrypted mail, both you and your recipient must have GPG keys set up and exchanged:

1. **Generate Keys:** Both parties must have generated a public/private key pair using GPG (as discussed previously, using `gpg --full-generate-key`).
2. **Exchange Public Keys:** You need the recipient's **Public Key**, and they need your Public Key. You can exchange these by:
 - **Exporting** the key: `gpg --armor --export 'Recipient Name' > recipient_key.asc` and sending the `.asc` file.
 - **Uploading** the key to a public key server.
3. **Import Key:** You must import the recipient's key into your GPG keyring: `gpg --import recipient_key.asc`.

3.2 Sending the Encrypted Email

The most common and user-friendly way to send GPG-encrypted emails on Ubuntu is by using **Mozilla Thunderbird** with the **Enigmail** add-on (or its built-in equivalent in modern versions of Thunderbird).

3.2.1 1. Compose the Message

- **Open Thunderbird** and start composing a new email.
- Write your message as usual.

3.2.2 2. Encryption and Signing

You will use the GPG function built into the mail client to perform two critical steps:

1. **Encryption:** You must encrypt the email using the **recipient's Public Key**. Only their corresponding **Private Key** can decrypt it. If you have multiple recipients, you must encrypt the message using the Public Key of *every single recipient*.
2. **Digital Signature:** You **sign** the email using **your Private Key**. This allows the recipient to verify that the email truly came from you and has not been tampered with in transit.

In Thunderbird, this is typically done by clicking a dedicated **OpenPGP or Security** menu or button within the compose window and ensuring both the "**Encrypt**" and "**Sign**" options are checked.

3.2.3 3. Verification and Sending

- The client will check that you have the required **Public Key** for the recipient(s). If a key is missing, it will warn you.
- When you click **Send**, Thunderbird uses GPG to encrypt the message body and attach your digital signature before transmitting the scrambled data.

3.2.4 4. Recipient's Experience (Decryption)

1. The recipient receives the scrambled email.
2. Their email client automatically uses their **Private Key** (protected by their passphrase) to decrypt the message contents, revealing the original text.
3. Their client simultaneously uses your **Public Key** to verify the digital signature, confirming the email's authenticity.

4 Privacy Tools From Prism Break

4.0.1 1. Tor Browser (Web Browsers / Anonymizing Networks)

- **What it is:** A web browser built on Firefox that routes your internet traffic through the Tor network, a volunteer-operated network of relays.
- **Privacy Focus:** Provides **strong anonymity** by obscuring your IP address and location from the websites you visit. It also includes anti-fingerprinting measures.

- **PRISM Break Note:** PRISM Break strongly recommends using Tor Browser for all web surfing when maximum anonymity is required.

4.0.2 2. Debian (Operating Systems)

- **What it is:** A popular and highly ethical GNU/Linux distribution known for its strict adherence to Free Software principles and ethical manifesto.
- **Privacy Focus:** Unlike proprietary operating systems like Windows and macOS (which PRISM Break generally avoids), Debian is fully open-source, allowing for audits. It has a long tradition of software freedom and transparency.
- **PRISM Break Note:** It's recommended as a top GNU/Linux choice for users transitioning from proprietary systems, highlighting its commitment to free software and its stable nature.

4.0.3 3. Thunderbird (Email Clients)

- **What it is:** A free, open-source, and cross-platform email client developed by Mozilla.
- **Privacy Focus:** Thunderbird is the top choice for desktop email due to its open-source nature and its long-standing **native support for OpenPGP** (GPG) encryption and digital signatures. This allows users to easily encrypt and authenticate their emails end-to-end.
- **PRISM Break Note:** It is highly recommended for securely managing email with built-in PGP features.

4.0.4 4. KeePassXC (Password Managers)

- **What it is:** A free, open-source, and cross-platform password manager.
- **Privacy Focus:** It stores all your passwords in a single, highly encrypted database file that is stored **locally** on your device, giving you total control over your sensitive data. It does not rely on a cloud service.
- **PRISM Break Note:** It is preferred for its strong encryption, open-source license, and local-only storage, minimizing exposure to third-party services.

4.0.5 5. Firefox (Web Browsers)

- **What it is:** A fast, flexible, and secure web browser developed by the non-profit Mozilla Foundation.

- **Privacy Focus:** Firefox is open-source and provides extensive privacy controls, including enhanced tracking protection (ETP), container technology, and a robust add-on ecosystem for further hardening security (like uBlock Origin).
- **PRISM Break Note:** While Tor Browser is for anonymity, Firefox is the recommended alternative for general web use when a site doesn't work well with Tor, provided the user configures its settings and replaces the default search engine with a privacy-focused one.

5 Open Source License

Certainly. Here is the information about the **MIT License** organized into clear, descriptive headings, strictly maintaining a paragraph-only format within each section.

5.1 The Core Purpose and Classification

The MIT License is renowned as one of the most permissive and concise open-source licenses currently in use. Originating from the Massachusetts Institute of Technology, its primary goal is to encourage maximum adoption and reuse of software with minimal legal friction. It is formally classified as a **permissive license**, meaning it grants users broad rights to use, modify, and distribute the software without imposing the reciprocal sharing obligations seen in copyleft licenses, such as the GNU General Public License (GPL). This makes the MIT License highly favorable for both commercial enterprises and proprietary software development.

5.2 Granted Rights and Permissions

The license grants blanket permission to any individual or entity obtaining a copy of the software and its associated documentation to deal with the Software without restriction. Specifically, users are granted explicit rights to **use, copy, modify, merge, publish, distribute, sublicense, and/or sell** copies of the software. This expansive grant allows developers to incorporate MIT-licensed code into projects that may ultimately be closed-source and sold commercially, provided they meet the few mandated conditions.

5.3 The Only Two Conditions for Distribution

Unlike licenses that enforce reciprocal sharing, the MIT License has only two critical requirements that must be met when the software is distributed or included in a larger work. The first condition is the mandatory inclusion of the original **Copyright Notice** (e.g., **Copyright <YEAR> <COPYRIGHT HOLDER>**).

The second is the mandatory inclusion of the full **License Text** itself. If these two simple requirements are satisfied, the user can otherwise treat the code as they wish, including releasing their modifications under a proprietary license.

5.4 Disclaimer of Warranty and Liability

A key component of the MIT License is its comprehensive liability disclaimer, which serves to protect the original authors. The license emphatically states that the software is provided **”AS IS,”** meaning it comes without any guarantee or warranty of any kind, whether express or implied, including warranties of merchantability or fitness for a particular purpose. Furthermore, the license explicitly protects the authors and copyright holders, asserting they **shall not be held liable** for any claim, damages, or other liability arising from the use or other dealings in the software. This places the entire risk associated with the software onto the end-user.

6 Self Hosted Server

6.1 About

FreshRSS is a free, open-source, self-hosted RSS feed aggregator that allows users to collect and read updates from multiple websites, blogs, YouTube channels, and online publications in one unified dashboard.

Instead of visiting individual websites, FreshRSS automatically fetches new content via RSS or Atom feeds and displays them in a clean, ad-free, chronological interface. It works entirely on your own server — giving you full control, privacy, and offline access to your subscribed content.

6.1.1 Key Features

Key Features:

- Collects and displays news, blog posts, and YouTube updates from multiple sources in one place.
- Supports RSS and Atom feeds.
- Works completely offline after fetching feeds.
- Simple and clean web interface with categories and search.
- Multi-user support with login system.
- Lightweight and fast — can run even on low-end systems.
- Can be accessed from any browser on the same network.
- Automatically updates feeds at regular intervals.

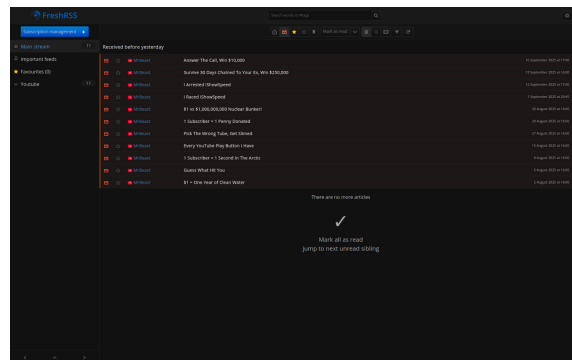


Figure 1: FreshRss Feed Reader

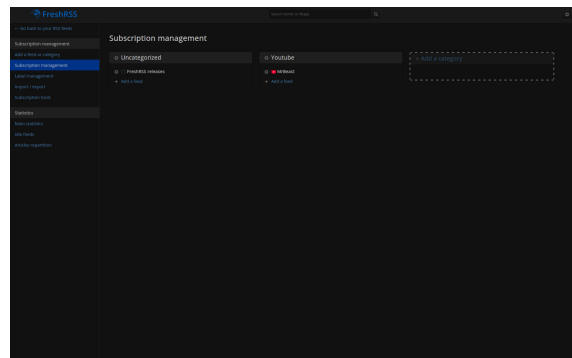


Figure 2: FreshRss Feed Reader

6.2 Installation Process (Docker Compose)

To install FreshRSS on my system, I followed a simple step-by-step process using Docker. First, I installed Docker and Docker Compose by running the command:

Installed Docker

```
curl -fsSL https://get.docker.com — sh
```

Then I added my user to the Docker group and logged in again.

Verified the installation

```
docker --version docker compose version
```

Created a project folder for FreshRSS

```
mkdir /freshrss cd /freshrss
```

Created the Docker Compose file I opened the file using:

```
nano docker-compose.yml
```

Then I added the FreshRSS service configuration with port 8080, data volumes, and auto-restart.

Started FreshRSS using Docker Compose

```
docker compose up -d
```

This downloaded the FreshRSS image and launched the container in the background.

Opened FreshRSS in the browser I went to:

```
http://localhost:8080
```

and completed the setup wizard by choosing SQLite as the database and creating my admin account.

Added RSS and YouTube feeds After logging in, I started adding feed URLs to receive updates inside FreshRSS.

This method allowed me to self-host FreshRSS quickly and efficiently, with automatic startup enabled and all data stored locally.

7 Open Source Contribution

7.1 PR 1 : LinkedIn Cruise Control

7.1.1 Goal

To enable CI/CD builds for the ppc64le architecture, as requested in issue [Enable cruise-control on ppc64le arch via CI/CD 1486](#). This PR adds 'ppc64le' to the *hw_platformmatrixwithinthebuild – platformjobinthe.github/workflows/ci.yaml* file.

Changes Done

File modified:

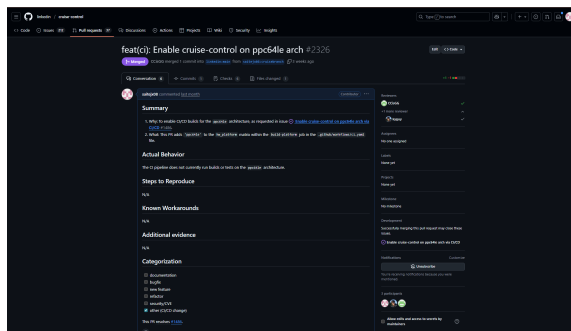


Figure 3: PR 1

.github/workflows/ci.yaml GitHub

Exact change:

In the build-platform job's strategy.matrix.hw_{platform}, the hardware platform list was updated from :

yaml : hw_{platform} : ['s390x']

to: yaml: hw_{platform} : ['s390x', 'ppc64le']

So effectively: 1 Added ppc64le to the hw_{platform} matrix. 2 This makes the CI workflow run the same build steps for both

7.2 PR 2: adding interactive examples to regex lesson

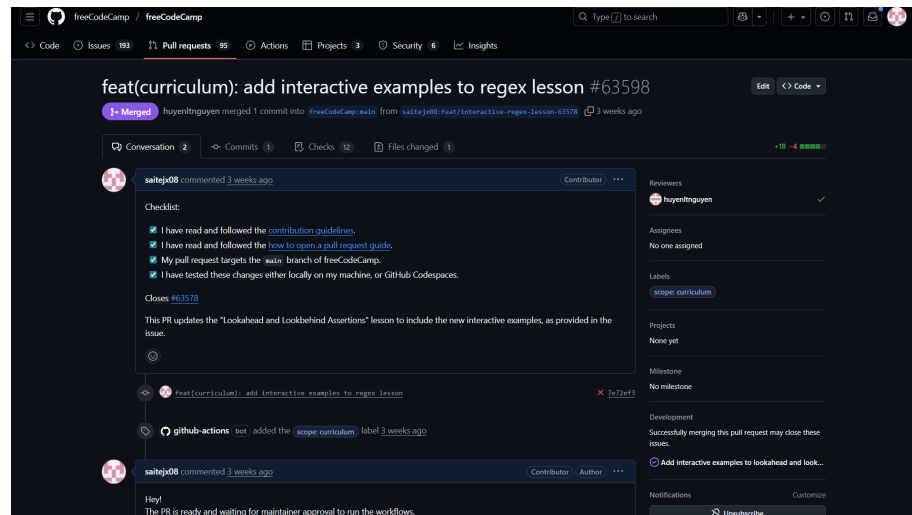
The pull request is titled “feat(curriculum): add interactive examples to regex lesson”. It enhances the curriculum by adding interactive examples to the “Lookahead and Lookbehind Assertions” section in the Regex lesson, making the content more hands-on for learners.

Changes Made

Added new interactive examples to the “Lookahead and Lookbehind Assertions” lesson in the curriculum.

Updated the lesson content files under the curriculum directory to include those interactive parts (exact files changed are listed under “Files changed” in the PR).

The PR also references closing issue 63578, linking the enhancement with a prior issue request.



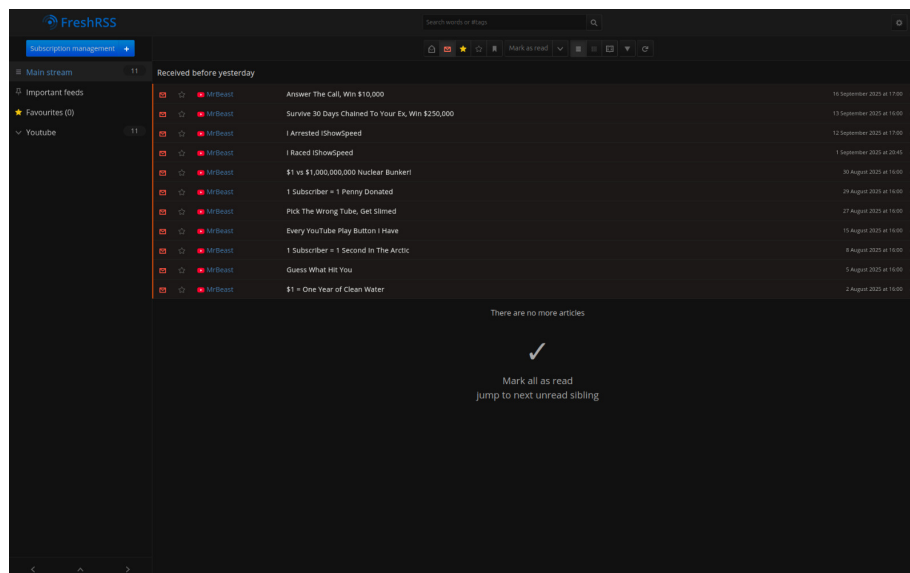
7.3 PR 3 : Y24 Open Source Engineering

I have kept a PR in this repo about my self hosting server. I installed and configured FreshRSS, which is a self-hosted RSS feed reader. It lets me collect and read updates from different websites, blogs, and YouTube channels all in one place. Instead of visiting each site separately, FreshRSS automatically fetches new articles or videos through their RSS links and displays them in a clean dashboard.

I hosted it locally using Docker because it's easy and fast to set up. I created a project folder, added a simple docker-compose.yml file, and started the container with docker compose up -d. Then I opened it in my browser at <http://localhost:8080> and completed the setup by choosing SQLite as the database and creating my admin account.

After setup, I added a few RSS links, including some YouTube channel feeds. Now all the new posts and videos appear directly inside FreshRSS, and it updates automatically. I also enabled the auto-start option in Docker so that FreshRSS runs automatically every time my system starts.

In short, FreshRSS helps me manage and read all my favorite content from one simple interface — fast, private, and completely under my control.



7.4 PR 4: First Contributors

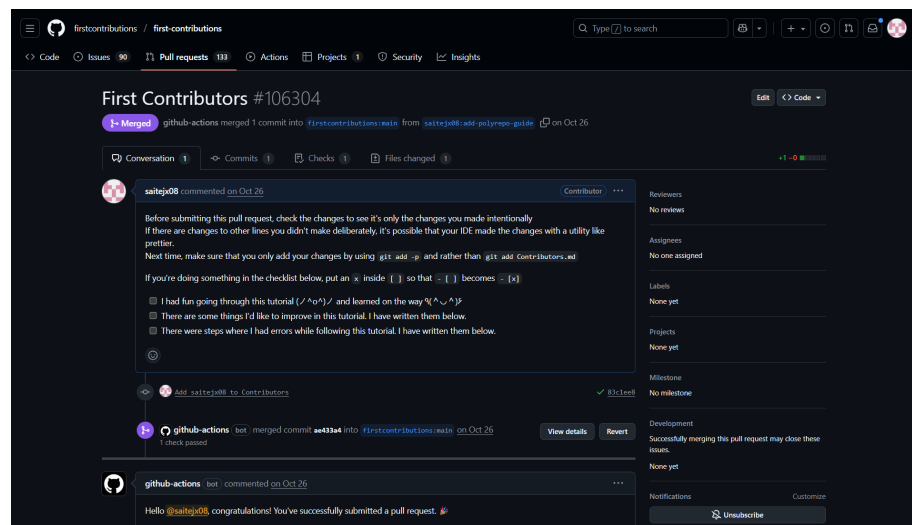
The pull request adds my name in First Contributors repo and it is a contribution. This pull request adds a new contribution to the repository. The contributor submitted a guide titled “add-polyrepo-guide” (branch saitejx08/add-polyrepo-guide) and requested that this be merged into the main branch. The overall purpose is to improve the repository by including this new guide resource.

Changes Made

A single commit was merged into main from the contributor’s branch. GitHub

The contributor’s guide (“polyrepo guide”) was added to the repository.

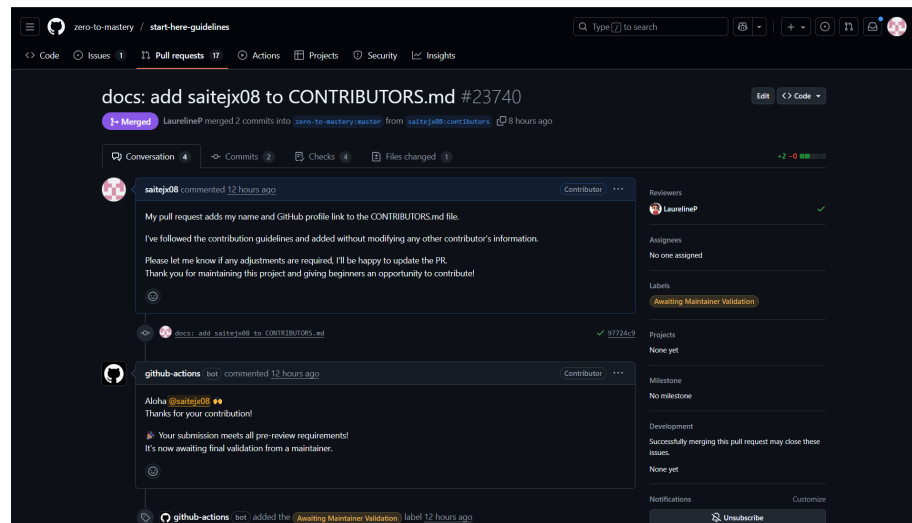
Likely modification to the file Contributors.md to add the contributor’s name, though exact file details are not explicitly visible in the summary view.



7.5 PR 5: start-here-guidelines

docs: add saitejx08 to CONTRIBUTORS.md My pull request adds my name and GitHub profile link to the CONTRIBUTORS.md file. The contributor submitted a guide titled “start-here-guidelines” (branch saitejx08/start-here-guidelines) and requested that this be merged into the main branch. The overall purpose is to improve the repository by including this new guide resource.

A lot of contributors have added their names to achieve contributions



8 LinkedIn Post Links

8.1 PR :

https://www.linkedin.com/posts/a-sai-teja-7b07a6345_opensource-developer-contribution-activity-73991026kc?utm_source=share&utm_medium=member_android&rcm=ACoAAFZvuvQBW9rxooeoUOIFZuNCNEQxXl026kc

8.2 Self Hosted Project :

https://www.linkedin.com/posts/a-sai-teja-7b07a6345_selfhosting-opensource-linuxserver-activity-73991026kc?utm_source=share&utm_medium=member_android&rcm=ACoAAFZvuvQBW9rxooeoUOIFZuNCNEQxXl026kc

8.3 Open Source Journey :

MyOpenSourceJourneyExperiencehttps://www.linkedin.com/pulse/my-opensource-journey-experience-a-sai-teja-7b07a6345?utm_source=share&utm_medium=member_android&utm_campaign=share_via